

6. CCS com passagem de valores

José Proença

(slides mainly from Nelma Moreira)

Concurrent programming (CC3040) 2025/2026

CISTER – U.Porto, Porto, Portugal

<https://fm-dcc.github.io/cp2526>

O modelo básico

$$\textit{Machine} \quad := \quad \textit{coin?}.\textit{coffee!}.\textit{Machine}$$

é muito simplista. Normalmente temos de usar várias moedas

$$\begin{aligned} \textit{Machine} &:= \textit{coin?}.\textit{Machine} + \textit{coin?}.\textit{PaidMachine} \\ \textit{PaidMachine} &:= \textit{coffee?}.(\textit{change!}.\textit{Machine} + \tau.\textit{Machine}) \end{aligned}$$

Mas, suponham uma máquina em que o café custa 5 euros e aceita moedas de 1, 2 e 5 euros. Como tomar café? Solução: considerar todas as possibilidades... e troco

Mas, suponham uma máquina em que o café custa 5 euros e aceita moedas de 1, 2 e 5 euros. Como tomar café? Solução: considerar todas as possibilidades... e troco

Machine0 := *coin1?.Machine1* + *coin2?.Machine2* + *coin5?.Machine5*

Machine1 := *coin1?.Machine2* + *coin2?.Machine3* + *coin5?.Machine6*

Machine2 := *coin1?.Machine3* + *coin2?.Machine4* + *coin5?.Machine7*

Machine3 := *coin1?.Machine4* + *coin2?.Machine5* + *coin5?.Machine8*

Machine4 := *coin1?.Machine5* + *coin2?.Machine6* + *coin5?.Machine9*

Machine5 := *coffee?.Machine0*

Machine6 := *coffee?.change1!.Machine0*

Machine7 := *coffee?.change1!.change1!.Machine0*

Machine8 := *coffee?.change1!.change1!.change1!.Machine0*

Machine9 := *coffee?.change1!.change1!.change1!.change1!.Machine0*

Mas, suponham uma máquina em que o café custa 5 euros e aceita moedas de 1, 2 e 5 euros. Como tomar café? Solução: considerar todas as possibilidades... e troco

Machine0 := *coin1?.Machine1* + *coin2?.Machine2* + *coin5?.Machine5*

Machine1 := *coin1?.Machine2* + *coin2?.Machine3* + *coin5?.Machine6*

Machine2 := *coin1?.Machine3* + *coin2?.Machine4* + *coin5?.Machine7*

Machine3 := *coin1?.Machine4* + *coin2?.Machine5* + *coin5?.Machine8*

Machine4 := *coin1?.Machine5* + *coin2?.Machine6* + *coin5?.Machine9*

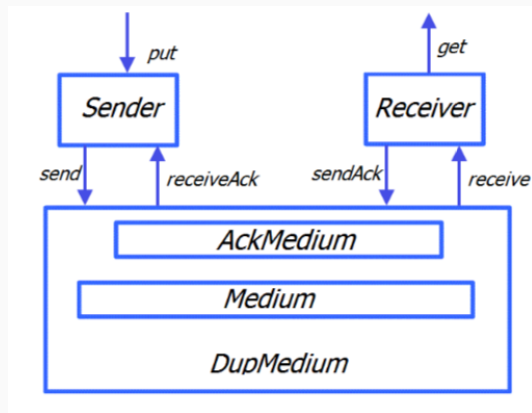
Machine5 := *coffee?.Machine0*

Machine6 := *coffee?.change1!.Machine0*

Machine7 := *coffee?.change1!.change1!.Machine0*

Machine8 := *coffee?.change1!.change1!.change1!.Machine0*

Machine9 := *coffee?.change1!.change1!.change1!.change1!.Machine0*



Sender := *put?.send!.Sending*

Sending := *receiveAck?.Sender + receiveNAck?.send!.Sending*

Receiver := *receive?.get?.sendAck!.Receiver +*
 gargled?.sendNAck!.Receiver

Medium := *send?.(receive!.Medium + i.garbled!.Medium)*

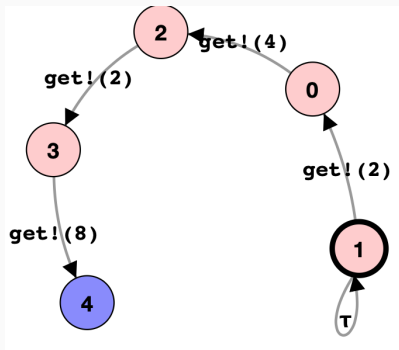
AckMedium := *sendAck?.receiveAck!.AckMedium +*
 sendNAck?.receivedNAck!.AckMedium

DupMedium := *Medium|AckMedium*

Protocol := (*Sender | Receiver | DupMedium*)\
 {*send, receive, sendAck, receiveAck,*
 receiveNAck, sendNAck, gargled}

$$\begin{aligned} \text{Sender} &:= \text{put?x.send!x.Sending}[x] \\ \text{Sending}[x] &:= \text{receiveAck?.Sender} + \text{receiveNAck?.send!x.Sending}[x] \\ \text{Receiver}[x] &:= \text{receive?x.get?x.sendAck!.Receiver}[x] + \\ &\quad \text{gargled?.sendNAck!.Receiver}[x] \\ \text{Medium} &:= \text{send?x.(receive!x.Medium} + \text{i.garbled!.Medium)} \\ \text{AckMedium} &:= \text{sendAck?.receiveAck!.AckMedium} + \\ &\quad \text{sendNAck?.receivedNAck!.AckMedium} \\ \text{DupMedium} &:= \text{Medium} | \text{AckMedium} \\ \text{Protocol} &:= (\text{Sender} | \text{Receiver} | \text{DupMedium}) \backslash \\ &\quad \{ \text{send}, \text{receive}, \text{sendAck}, \text{receiveAck}, \\ &\quad \text{receiveNAck}, \text{sendNAck}, \text{garbled} \} \end{aligned}$$

Protocol | *put!2.put!4.put!2.put!8.0* \ { *put* }



- $a!v$: saída do valor v no canal a (enviar)

- $a!v$: saída do valor v no canal a (enviar)
- $a?v$: entrada do valor v pelo canal a (receber)

- $a!v$: saída do valor v no canal a (enviar)
- $a?v$: entrada do valor v pelo canal a (receber)
- ou usar variáveis

- $a!v$: saída do valor v no canal a (enviar)
- $a?v$: entrada do valor v pelo canal a (receber)
- ou usar variáveis
- $a!x$: saída do valor guardado em x no canal a (enviar)

- $a!v$: saída do valor v no canal a (enviar)
- $a?v$: entrada do valor v pelo canal a (receber)
- ou usar variáveis
- $a!x$: saída do valor guardado em x no canal a (enviar)
- $a?x$: entrada de um valor que se guarda em x pelo canal a (receber)

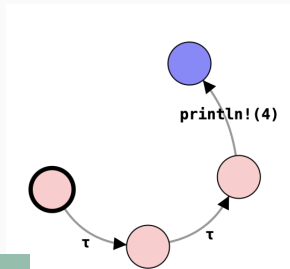
- $a!v$: saída do valor v no canal a (enviar)
- $a?v$: entrada do valor v pelo canal a (receber)
- ou usar variáveis
- $a!x$: saída do valor guardado em x no canal a (enviar)
- $a?x$: entrada de um valor que se guarda em x pelo canal a (receber)
- e os nomes dos processos podem ter variáveis com parâmetros permitindo assim enviar e receber valores ($A[x, y]$)

$$\begin{aligned} B &:= put?x.B1[x] \\ B1[x] &:= get!(x + 1).B \end{aligned}$$

Então temos para

$$(B|put!3.get?y.println!y)\{*, println\}$$

o LTS



Sendo $\mathbb{V}$ um conjunto de valores e $\mathbb{K}$ um conjunto de canais temos

Sendo $\mathbb{V}$ um conjunto de valores e $\mathbb{K}$ um conjunto de canais temos

$$A^! = \{a!v \mid a \in \mathbb{K}, v \in \mathbb{V}\} \cup \{a! \mid a \in \mathbb{K}\},$$

$$A^? = \{a?v \mid a \in \mathbb{K}, v \in \mathbb{V}\} \cup \{a? \mid a \in \mathbb{K}\},$$

$$Com = A^! \cup A^?$$

$$Act = Com \cup \{\tau\}$$

Sendo $\mathbb{V}$ um conjunto de valores e $\mathbb{K}$ um conjunto de canais temos

$$A^! = \{a!v \mid a \in \mathbb{K}, v \in \mathbb{V}\} \cup \{a! \mid a \in \mathbb{K}\},$$

$$A^? = \{a?v \mid a \in \mathbb{K}, v \in \mathbb{V}\} \cup \{a? \mid a \in \mathbb{K}\},$$

$$Com = A^! \cup A^?$$

$$Act = Com \cup \{\tau\}$$

$$P ::= 0 \mid X[r_1, \dots, r_n] \mid P + P \mid \alpha.P \mid P|P \mid P \setminus H$$

$$\alpha ::= \tau \mid a! \mid a? \mid a!v \mid a?v \mid a!x \mid a?x$$

onde $X \in Var$, $x \in D$, $r_i \in D \cup \mathbb{V} \cup \mathbb{K}$

$$\text{Pref} \frac{\alpha \in Act}{\alpha.P \xrightarrow{\alpha} P}$$

$$\text{Pref} \frac{\alpha \in Act}{\alpha.P \xrightarrow{\alpha} P}$$

A avaliação de expressões e $\Downarrow z$: a expressão é avaliada para z .

$$\text{Pref} \frac{\alpha \in \text{Act}}{\alpha.P \xrightarrow{\alpha} P}$$

A avaliação de expressões e $\Downarrow z$: a expressão e avalia para z.

$$\text{Output} \frac{e \Downarrow z}{a!e.P \xrightarrow{a!z} P}$$

$$\text{Valor} \frac{e \Downarrow z}{a?e.P \xrightarrow{a?z} P}$$

$$\text{Input} \frac{v \in \mathbb{V}}{a?x.P \xrightarrow{a?v} P\{v/x\}}$$

onde $P\{v/x\}$ é P onde x é substituído por v (e x não é uma ação)

$$\text{Pref} \frac{\alpha \in \text{Act}}{\alpha.P \xrightarrow{\alpha} P}$$

$$\text{Input} \frac{v \in \mathbb{V}}{a?x.P \xrightarrow{a?v} P\{v/x\}}$$

onde $P\{v/x\}$ é P onde x é substituído por v (e x não é uma ação)

$$(a!y.P)\{v/x\} = a!y.P\{v/x\} \quad \text{se } y \neq x$$

$$(a!x.P)\{v/x\} = a!v.P\{v/x\}$$

$$(a?y.P)\{v/x\} = a?y.P\{v/x\} \quad \text{se } y \neq x$$

$$(a?x.P)\{v/x\} = a?x.P$$

$$X[x]\{v/x\} = X[v]$$

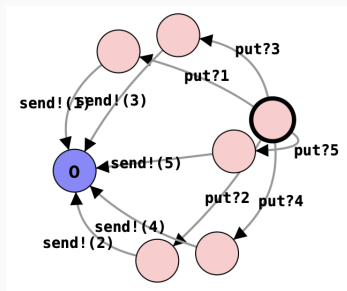
$$X[y]\{v/x\} = X[y] \quad \text{se } y \neq x$$

Para as restantes expressões é passado para as subexpressões.

$$\begin{aligned} \text{send!}y.\text{Sending}[x]\{3/x, 5/y\} &= \text{send!}5.\text{Sending}[3] \\ \text{send!}y.\text{Sending}[x]\{3/x, 5/y, \text{receive/send}\} &= \text{receive!}5.\text{Sending}[3] \end{aligned}$$

$$\begin{aligned} \text{send!}(x + y)\{3/x, 5/y\} &= \text{sent!}(8) \\ 3 + 5 &\Downarrow 8 \end{aligned}$$

Calcular o LTS de $put?x : 0..5.send!x.0$



$$\text{Rec} \frac{P\{v_1/r_1, \dots, v_n/r_n\} \xrightarrow{\alpha} P' \quad \Gamma(X[r_1, \dots, r_n]) = P}{X[v_1, \dots, v_n] \xrightarrow{\alpha} P'}$$

Supomos $\mathbb{V} = \mathbb{Z} (CCS_{vp}^Z)$

$$B[x] \quad := \quad when(x < 4)put?.B[x + 1] + when(x > 0)get?.B[x - 1]$$

$B[0]$ ou $B[5]$ o que fazem?

Supomos $\mathbb{V} = \mathbb{Z} (CCS_{vp}^Z)$

A expressão $when(b)P$ se b é verdade comporta-se como P senão bloqueia.

Vamos só considerar expressões com inteiros.

$$\text{cond} \frac{P \xrightarrow{\alpha} P' \quad b \Downarrow \text{True}}{when(b)P \xrightarrow{\alpha} P'}$$

Acrescentamos à gramática

$$\begin{aligned} P &::= 0 \mid X[r_1, \dots, r_n] \mid P + P \mid \alpha.P \mid P \mid P \mid P \setminus H \mid \text{when}(b)P \\ \alpha &::= \tau \mid a! \mid a? \mid a!v \mid a?v \mid a!x \mid a?x \end{aligned}$$

$$\begin{aligned}
Machine[b] &:= when(b < 5) coin?c.Machine[b + c] \\
&\quad + when(b \geq 5) coffee!.ReturnMachine[b - 5] \\
ReturnMachine[b] &:= when(b > 0) change!.ReturnMachine[b - 1] \\
&\quad + Machine[0] \\
User &:= coin!2.coin!2.coin!2.coffee?.change?.0
\end{aligned}$$

Calcula $\llbracket (Machine[0] \parallel User) \setminus \{coin, change, coffee\} \rrbracket_{\Gamma}$.

IterMult[*z*, *x*, *y*] := *when*(*x* > 0)*i.IterMult*[*z* + *y*, *x* − 1, *y*]
+ *when*(*x* == 0)*println!**z*.0

IterMult[0, 3, 7]

$$\text{Pref} \frac{\alpha \in \text{Act}}{\alpha.P \xrightarrow{\alpha} P}$$

$$\text{Input} \frac{v \in \mathbb{V}}{a?x.P \xrightarrow{a?v} P\{v/x\}}$$

$$\text{Rec} \frac{P\{v_1/r_1, \dots, v_n/r_n\} \xrightarrow{\alpha} P' \quad \Gamma(X[r_1, \dots, r_n]) = P}{X[r_1, \dots, r_n] \xrightarrow{\alpha} P'}$$

$$\text{Output} \frac{e \Downarrow z}{a!e.P \xrightarrow{a!.z} P}$$

$$\text{Valor} \frac{e \Downarrow z}{a?e.P \xrightarrow{a?z} P}$$

$$\text{cond} \frac{P \xrightarrow{\alpha} P' \quad b \Downarrow \text{True}}{\text{when}(b)P \xrightarrow{\alpha} P'}$$

$$\begin{aligned} \text{Fac}[n, j] &:= \text{when}(j > 0) i. \text{Fac}[n * j, j - 1] \\ &\quad + \text{when}(j == 0) \text{println!} n. 0 \end{aligned}$$

Calcular $\text{Fac}[1, 5]$

$$Cell_x[cur] := get_x!cur.Cell_x[cur] + set_x?new : 0..2.Cell_x[new]$$

Mais geralmente:

$$Cell[rd, wr, x] := rd!x.Cell[rd, wr, x] + wr?y.Cell[rd, wr, y]$$

Mais geralmente:

$$\text{Cell}[rd, wr, x] := rd!x. \text{Cell}[rd, wr, x] + wr?y. \text{Cell}[rd, wr, y]$$

- $\text{Cell}[rd, wr, 5]$
- $\text{Cell}[rdA, wrA, 0] \mid \text{Cell}[rdB, wrB, 0]$

Mais geralmente:

$$\text{Cell}[rd, wr, x] := rd!x.\text{Cell}[rd, wr, x] + wr?y.\text{Cell}[rd, wr, y]$$

- $\text{Cell}[rd, wr, 5]$
- $\text{Cell}[rdA, wrA, 0] | \text{Cell}[rdB, wrB, 0]$

$$\text{Cells} := \text{Cell}[rdA, wrA, 0] | \text{Cell}[rdB, wrB, 0]$$

$$\text{Serve} := \text{mult}?.rdA?x : R.rdB?y : R.\text{IterMult}[0, x, y]$$

$$\begin{aligned} \text{IterMult}[z, x, y] &:= \text{when}(x > 0)i.\text{IterMult}[z + y, x - 1, y] \\ &\quad + \text{when}(x == 0)\text{println!}z.\text{Serve} \end{aligned}$$

$$\text{Use} := wrA!7.wrB!5.\text{mult}!.0$$

$$(\text{Cells} | \text{Serve} | \text{Use}) \setminus \{rdA, wrA, rdB, wrB, \text{mult}\}$$

$$\begin{aligned} \text{Fac}[n, j] &:= \text{when}(j > 0) i. \text{Fac}[n * j, j - 1] \\ &\quad + \text{when}(j == 0) \text{println!} n. 0 \end{aligned}$$

Ficheiro no Pseuco.com:

<https://pseuco.com/#/edit/remote/5bglrm4937vze10dvlqi>

$$\begin{aligned} Fac[n, j] &:= \text{when}(j > 0) i. Fac[n * j, j - 1] \\ &\quad + \text{when}(j == 0) \text{println!} n. 0 \end{aligned}$$

$$\begin{aligned} Fak &:= rdJ?j : R. (\text{when}(j > 0) rdN?n. wrN!(n * j). wrJ!(j - 1). Fak \\ &\quad + \text{when}(j == 0) rdN?n. \text{print!} n. 0) \end{aligned}$$

$$Cell[v, rd, wr] := rd!v. Cell[v, rd, wr] + wr?x : R. Cell[x, rd, wr]$$

$$Cells := Cell[0, rdN, wrN] | Cell[0, rdJ, wrJ]$$

$$(wrN!1. wrJ!5. Fak | Cells) \setminus \{rdN, wrN, rdJ, wrJ\}$$

..logo é só "syntactic sugar"...

CCS_{vp}	CCS
$a!v.P$	$a_v!.P$
$a?x.P$	$\sum_{v \in \mathbb{V}} a_v?.P\{v/x\}$
$X[u_1, \dots, u_n]$	$X_{u_1, \dots, u_n}$

Isto é basta usar ações e nomes indexados, podendo ser considerados conjuntos infinitos de índices ($\mathbb{V}$ ou D)

O seguinte código

```
while (a > 0) {
  println("loop");
  a = a-1;
}
println("a is zero");
```

pode ser escrito em CCS

```
P[a] := when(a ≤ 0) i.Q[a] +
        when(a > 0) println!"loop" . P[a-1]
Q[a] := println!"a is zero"
```

$P[3]$ teria o seguinte LTS

